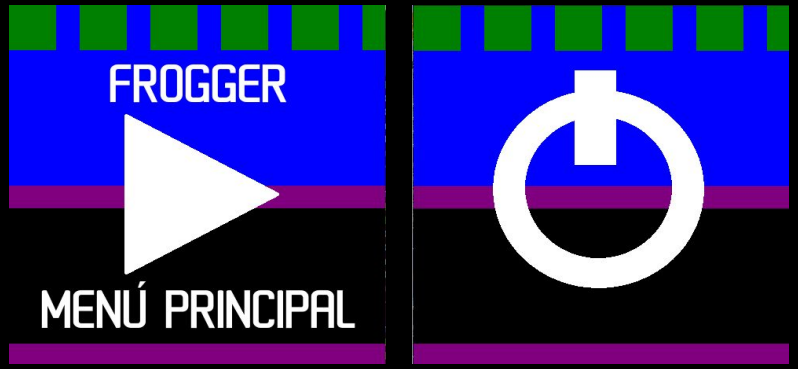
FROGGER

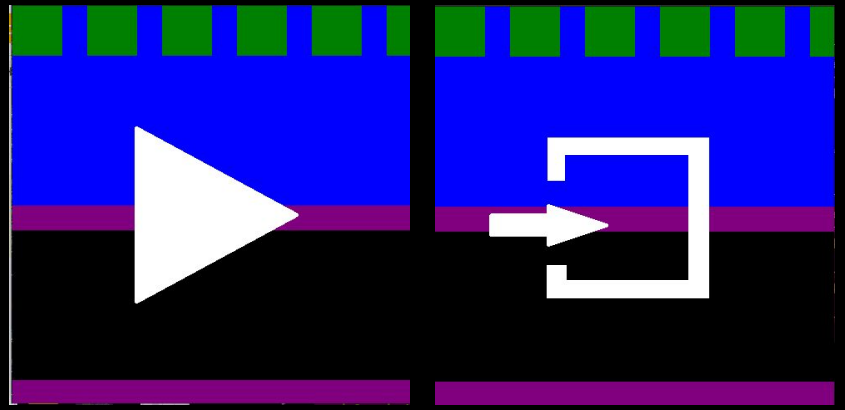
Presentado por: Carlos Chen, Santiago Feldman, Candela Gioia, Tiago Nanni

FUNCIONALIDAD



MENÚ PRINCIPAL

MENÚ DE PAUSA



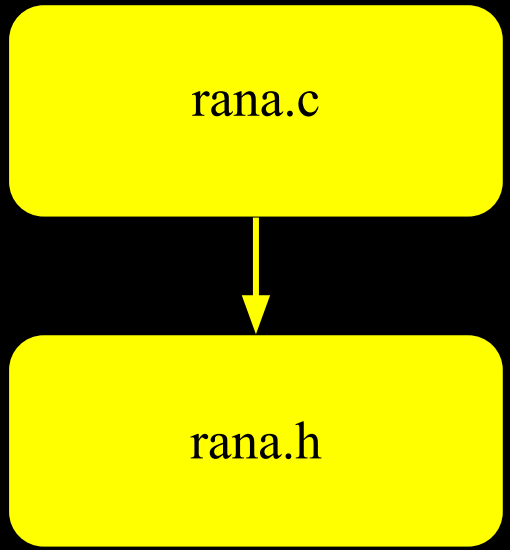
FUNCIONALIDAD

- El juego consiste en 3 niveles con distinta dificultad
- Por cada fila que se avanza se suman 10 puntos, por cada fila que se retrocede se restan 10 puntos y por cada charco que se completa se suma 100 puntos
- Cuando se completan los 3 niveles se vuelve al primer nivel con el score para poder seguir acumulándolo

BACK_END

- obj.c y obj.h: Inicializa los objetos/obstáculos y los mueve
- rana.c y rana.h: Inicializa la rana y la mueve
- colisiones.c y colisiones.h: Analiza las colisiones de la rana con el medio

BACK_END

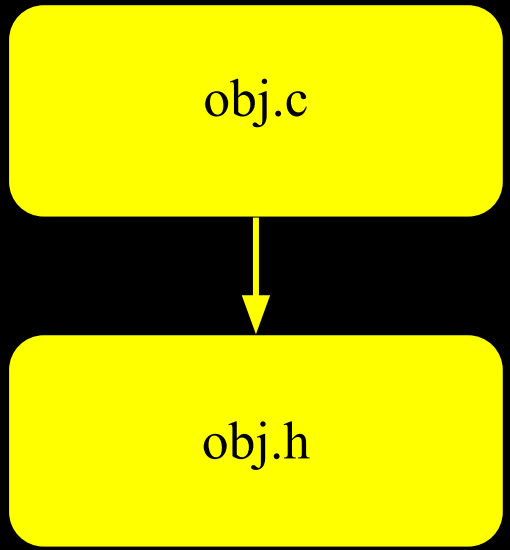


Funciones:

- `rana_t pos_rana_init(rana_t init_pos, int pasos)`
- `rana_t reset_rana(rana_t rana, int pasos)`
- `rana_t movimiento_rana(char tecla, rana_t posicion, int pasos, int *score)`

```
typedef struct
{
    int16_t pos_x;
    int16_t pos_y;
    uint8_t colision;
    uint8_t cant_vida;
}rana_t;
```

BACK_END



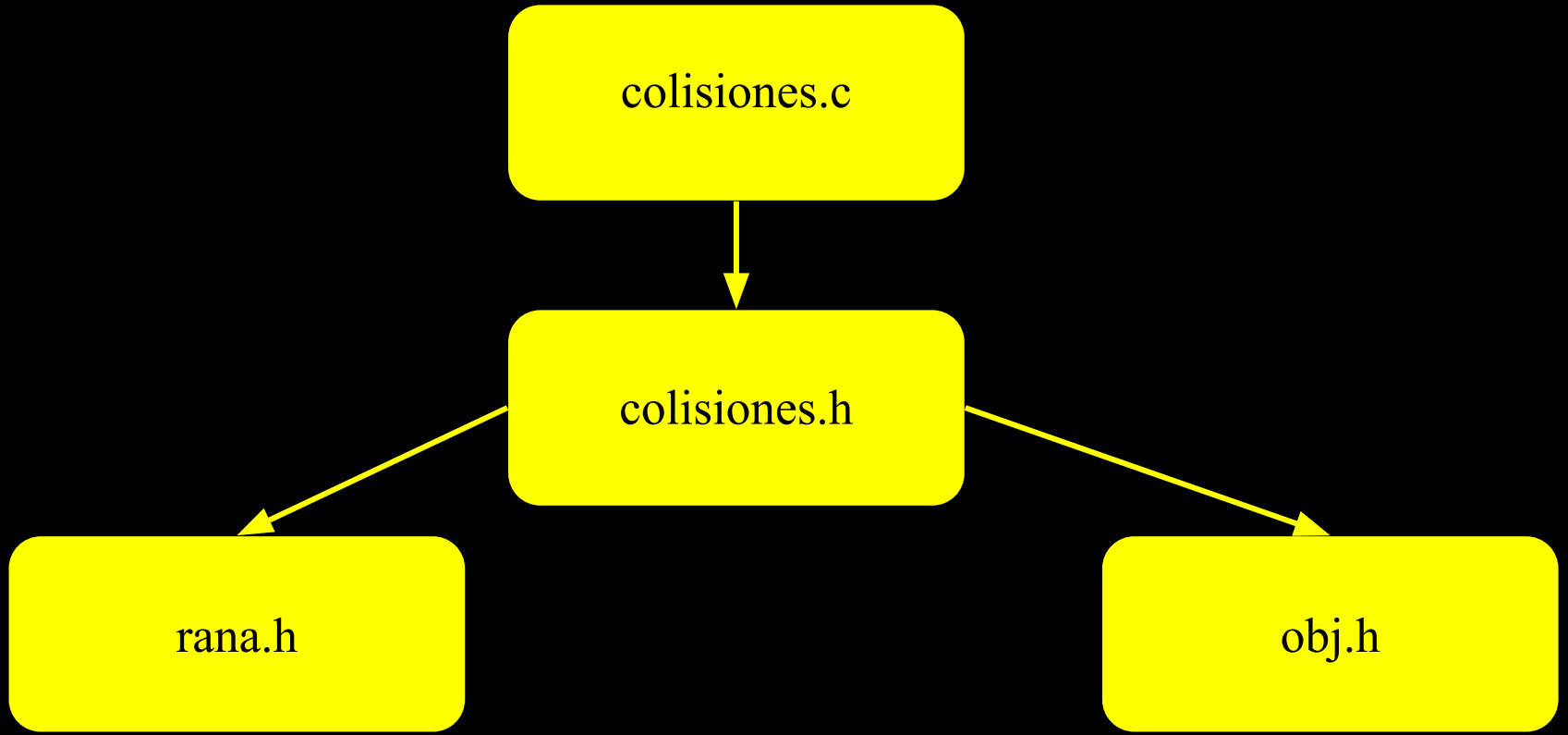
Funciones:

- `void init_autos(obj_t autos[], int pasos, uint8_t nivel, int speed)`
- `void init_troncos(obj_t troncos[], int pasos, uint8_t nivel, int speed)`
- `void init_charcos(charco_t charcos[], int pasos)`
- `obj_t movimiento_obj(obj_t objeto, int pasos)`

```
typedef struct
{
    int16_t pos;
    uint8_t filled;
} charco_t;
```

```
typedef struct
{
    int16_t pos_x;
    int16_t pos_y;
    int16_t width;
    uint8_t type_obj;
    int speed;
    int original_speed;
} obj_t;
```

BACK_END



BACK_END

Funciones globales:

- `rana_t colision (rana_t rana, int pasos, obj_t autos[], obj_t troncos[], int cant_au, int cant_tl);`
- `rana_t check_charcos(rana_t rana, charco_t charcos[], int pasos, int* nivel, int *score);`
- `int highscore_check (int score);`

Funciones static:

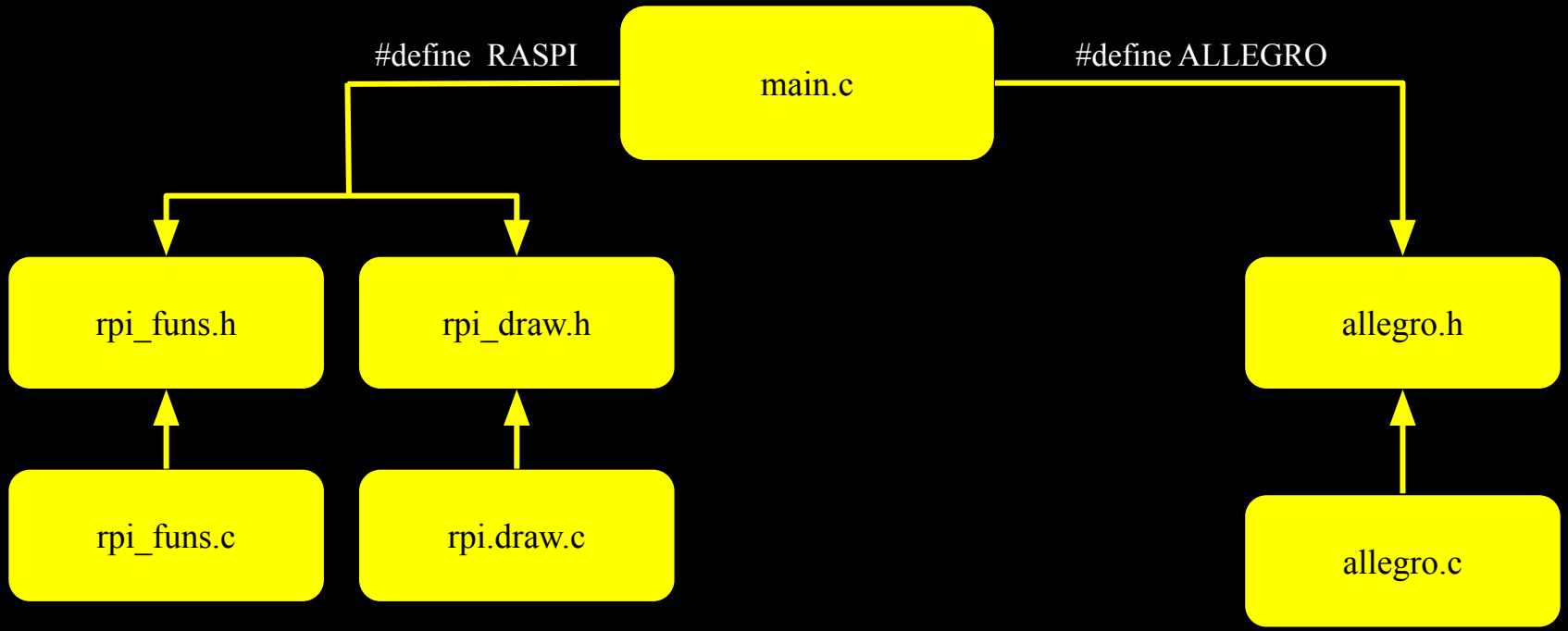
- `static rana_t movimiento_rana_tronco(rana_t rana, obj_t objeto , int pasos )`
- `static rana_t check_colision(rana_t rana, obj_t objeto, int pasos)`
- `static rana_t reset_life(rana_t rana, int pasos)`

```c
int highscore_check (int score)
{
    FILE *highscore_f;

    highscore_f = fopen("highscore.txt","r");

    int c = fgetc(highscore_f);
    int ent = 0;

    if(c == EOF)
    {
        fclose(highscore_f);
        highscore_f = fopen("highscore.txt","w");
        fprintf(highscore_f, "%d", 0);
        fclose(highscore_f);
    }
    else
    {
        while (c!= EOF)
        {
                ent *= 10;
                ent += c - ASCII;
                c = fgetc (highscore_f);
        }
        fclose(highscore_f);
    }
    if (score > ent )
    {
            highscore_f = fopen  ("highscore.txt","w");
            fprintf(highscore_f, "%d", score);
            fclose(highscore_f);
            ent = score;
    }

    return ent;
}
```

BACK_END

```
rana_t colision (rana_t rana, int pasos, obj_t autos[], obj_t troncos[], int cant_au, int cant_tl)
{

    int i;
    int conta_colisiones=0;
    if (rana.pos_y <= 7*pasos && rana.pos_y >= 2*pasos)
    {
        for (i=0; i<cant_tl; i++)
        {
            rana = check_colision(rana, troncos[i], pasos);
            conta_colisiones += rana.colision;
                        if (rana.colision == 0 )
                        {

                             rana = movimiento_rana_tronco( rana,troncos[i], pasos);
                             i = cant_tl;

                        }
                        else if (conta_colisiones == cant_tl)
                        {

                                rana = reset_life(rana,pasos);
                        }

        }

    }
        else
    {
            for (i=0; i<cant_au; i++)
        {
            rana = check_colision(rana, autos[i], pasos);

            if (rana.colision == 1)
            {
                rana = reset_life(rana,pasos);
                i=cant_au;
            }


        }
    }
    return rana;

}
```

FRONT_END


#define RASPI
main.c
#define ALLEGRO
rpi_funs.h
rpi_draw.h
allegro.h
rpi_funs.c
rpi.draw.c
allegro.c

ALLEGRO

Funciones globales:

- `pointers_t init_pointers (bool *do_exit)`
- `void draw_all ( int cantidad_au, int cantidad_tr,obj_t autos[], obj_t troncos[], pointers_t bits)`
- `void draw_rana (float x, float y, int direccion,ALLEGRO_BITMAP *ranabit_y, ALLEGRO_BITMAP*ranabit_x)`
- `void landscape(void)`
- `rana_t init_level (int nivel,obj_t autos[], obj_t troncos[], rana_t rana)`
- `rana_t ev_time (bool key_pressed[], pointers_t pointers, rana_t rana, int *score, int *direccion,bool *menu_flag, bool *pause_flag, bool *do_exit, int *select)`
- `void ev_tecla(bool state, bool key_pressed[], ALLEGRO_EVENT ev )`
- `void draw_menu (int select, pointers_t pointers, bool menu_flag)`
- `void draw_level(int nivel, ALLEGRO_FONT* font)`
- `void show_score (int score, ALLEGRO_FONT*font )`
- `void draw_lives( uint8_t cant_vida,int x, int y, int type, ALLEGRO_BITMAP* point, ALLEGRO_FONT* font )`

Funciones static:

- `static void draw_obt( int16_t x, int16_t y,ALLEGRO_BITMAP* objbit ,int typeobj, int16_t width)`
- `static ALLEGRO_BITMAP* sprite_grab(int x, int y, int w, int h,int foto)`

ALLEGRO

```
void draw_rana (float x, float y, int *direccion,ALLEGRO_BITMAP*ranabit_y, ALLEGRO_BITMAP*ranabit_x)
{

            if (*direccion == KEY_UP)
            {
                al_draw_scaled_bitmap(ranabit_y,
                0, 0, al_get_bitmap_width(ranabit_y), al_get_bitmap_height(ranabit_y),
                x,y, RANA_W*PASOS, RANA_H*PASOS,
                0);
            }
            else if(*direccion == KEY_DOWN)
            {
                al_draw_scaled_bitmap(ranabit_y,
                0, 0, al_get_bitmap_width(ranabit_y), al_get_bitmap_height(ranabit_y),
                x, y, RANA_W*PASOS, RANA_H*PASOS,
                ALLEGRO_FLIP_VERTICAL);

            }
            else if(*direccion == KEY_LEFT)
            {
                al_draw_scaled_bitmap(ranabit_x,
                0, 0, al_get_bitmap_width(ranabit_x), al_get_bitmap_height(ranabit_x),
                x, y, RANA_W*PASOS, RANA_H*PASOS,
                ALLEGRO_FLIP_HORIZONTAL);

            }
            else if(*direccion == KEY_RIGHT)
            {
                al_draw_scaled_bitmap(ranabit_x,
                0, 0, al_get_bitmap_width(ranabit_x), al_get_bitmap_height(ranabit_x),
                x, y, RANA_W*PASOS, RANA_H*PASOS,
                0);

            }
            else if (*direccion == DEATH)
            {
                *direccion = KEY_UP;
                al_draw_scaled_bitmap(ranabit_x,
                0, 0, al_get_bitmap_width(ranabit_x), al_get_bitmap_height(ranabit_x),
                x, y, RANA_W*PASOS, RANA_H*PASOS,
                0);
            }

            else
            {
                al_draw_scaled_bitmap(ranabit_y,
                0, 0, al_get_bitmap_width(ranabit_y), al_get_bitmap_height(ranabit_y),
                x, y, RANA_W*PASOS, RANA_H*PASOS,
                0);
            }
}
```

ALLEGRO

FUNCIÓN SPRITE_GRAB

```
static ALLEGRO_BITMAP* sprite_grab(int x, int y, int w, int h,int foto)
{
    ALLEGRO_BITMAP *sprites;

    if (foto == 1)
    {
        sprites  = al_load_bitmap("frogger_sprites.png");
    }
    else
    {
         sprites  = al_load_bitmap("death.png");
    }
    ALLEGRO_BITMAP* sprite = al_create_sub_bitmap(sprites, x, y, w, h);
    return sprite;
}
```

Raspberry Pi - rpi_funs.c

Funciones globales:

- `void init_agua(void);`
- `void init_charcos_rpi(charco_t charcos[]);`
- `void init_obj(obj_t autos[], int size_autos, obj_t troncos[], int size_troncos, int nivel);`
- `void move_obj(obj_t mat[], int size_mat, dlevel_t val);`

Funciones static:

- `static void se_fue(dcoord_t pos, int16_t width, int state);`

Raspberry Pi - rpi_draw.c

Funciones globales:

- `void menu(char* p_opt, void(*p2fun)(void));`
- `void vidas_restantes(int cant_vidas);`
- `void draw_skull(void);`
- `void draw_off(void);`
- `void draw_exit(void);`
- `void show_score(int score, int highscore);`

Funciones static:

- `static void draw_play(void);`
- `static void draw_heart(dcoord_t start);`
- `static void draw_cifras(char arr[], int score, dcoord_t start, char altura_y);`
- `static void draw_num_vida(int vida);`
- `static void draw_num(char num, dcoord_t start);`
- `static void draw_top(dcoord_t start);`
- `static void draw_score(dcoord_t start);`

Raspberry Pi

```
while((pause_flag != OFF)&&(break_flag != OFF))
{
    disp_update();
    move_obj(autos, cant_auto, D_OFF);
    move_obj(troncos, cant_tronco, D_ON);

    joy_update();
    coord = joy_get_coord();
    if((coord.x > THRESHOLD) && (npos.x <= DISP_MAX_X)) letra = 'R';
    if((coord.x < -THRESHOLD) && (npos.x > DISP_MIN)) letra = 'L';
    if((coord.y > THRESHOLD) && (npos.y > DISP_MIN)) letra = 'U';
    if((coord.y < -THRESHOLD) && (npos.y <= DISP_MAX_Y)) letra = 'D';
    rana = movimiento_rana(letra, rana, RPI, p_score);
    npos.x = rana.pos_x;
    npos.y = rana.pos_y;
    disp_write(pos,D_OFF);
    disp_write(npos,D_ON);
    pos = npos;
    letra = ON;

    int vida_antes = rana.cant_vida;
    rana = colision (rana, RPI, autos, troncos, cant_auto, cant_tronco);
    int nivel_antes = nivel;
    rana = check_charcos(rana, charcos, RPI, p_nivel, p_score);
```

Raspberry Pi

```
void menu(char* p_opt, void(*p2fun)(void))
{
    dcoord_t pos = {DISP_MAX_X>>1 , DISP_MAX_Y>>1};
    dcoord_t npos = pos;
    jcoord_t coord = {0,0};
    draw_play();
    int option_flag=PLAY;

    while(*p_opt == 0)
    {
        joy_update();
        coord = joy_get_coord();
        if((coord.x > THRESHOLD) && (npos.x <= DISP_MAX_X))
        {
            disp_clear();
            p2fun();
            option_flag = EXIT;
        }
        if((coord.x < -THRESHOLD) && (npos.x > DISP_MIN))
        {
            disp_clear();
            draw_play();
            option_flag = PLAY;
        }
        switch (option_flag)
        {
            case PLAY:
            {
                if(joy_get_switch()!=J_NOPRESS)
                {
                    *p_opt=PLAY;
                }
                break;
            }
            case EXIT:
            {
                if(joy_get_switch()!=J_NOPRESS)
                {
                    *p_opt=EXIT;
                }
                break;
            }
            default:
                break;
        }

    }
}
```

¡MUCHAS GRACIAS!